AssetGuard Complete System Documentation

Government asset tracking features architecture and operation

AssetGuard | Version 1.0.0 | 30 October 2026

Project overview

AssetGuard is a local web application for registering equipment, assigning responsibility, observing Bluetooth connectivity, raising alerts and recording an audit history. It now discovers Windows Bluetooth devices, imports detected devices into the asset registry and automatically registers connected devices. Administrators can also simulate transfers between facility zones. A local SHA-256 hash chain records asset lifecycle and tracking events.

For the demonstration, a mobile phone represents an asset and a laptop runs the backend and monitoring service. Administrators manage records and simulate movement; officers inspect the dashboard, history and ledger. React provides the interface, while Spring Boot handles authentication, validation, tracking rules and persistence.

The application is a local indoor demonstration. Bluetooth state does not establish a precise physical location, and the zone simulator records submitted locations rather than sensor measurements. The ledger checks the consistency of stored event data; it does not establish that a physical event really occurred.

What this document contains

Section	Coverage
1 to 3	Objectives and roles, architecture, and technology stack
4 to 5	Screens, features and end to end operating workflows
6 to 7	Database design and field dictionary
8 to 9	Session, query and mutation REST endpoints
10 to 12	Bluetooth monitor, audit ledger, and security controls
13 to 15	Build and operation, testing, limitations and next steps
16	Implementation map and terminology
17 to 18	Bluetooth discovery and import plus notification delivery

Current implementation position

The project contains React sources, Maven configuration, backend services, test definitions and build scripts. H2 is the default database and Windows is the default monitor mode. The Windows path uses a native API connection flag rather than the older PnP health check. Linux and demo modes remain available. Important gaps remain in scan error handling, import authorization, password reset behavior and audit guarantees.

1 Objectives scope and user roles

Problem being addressed

Departments need an organized record of laptops, phones and equipment, a named officer responsible for each device, a visible connection state and an event history. A monitoring station can help demonstrate how a connectivity loss triggers a review, while a movement simulator demonstrates permission rules for named facility zones.

Project objectives

- Maintain an asset registry with unique serial numbers and Bluetooth addresses, department assignments and responsible officers.
- Show the last observed connection state and create an alert when the state becomes disconnected.
- Record connection changes and location transfers with timestamps that are displayed in the browser local time.
- Compare a submitted destination with an asset specific list of permitted zones and flag unauthorized movement.
- Record asset lifecycle and tracking events in a local SHA-256 hash chain and provide an integrity checking interface.

Roles and permissions

Capability	Admin	Officer
Sign in and sign out	Yes	Yes
View registry dashboard alerts and history	Yes	Yes
Inspect ledger and run an integrity check	Yes	Yes
Register edit and delete assets	Yes	No
Submit a location transfer	Yes	No
Simulate connection state in demo mode	Yes	No
Import Bluetooth devices	Shown in UI	API currently permits import

Windows auto-discovery also registers connected devices without a user action. Import is shown only to admins, but POST /api/bluetooth/sync currently requires authentication rather than ADMIN. This is an authorization gap. The assignment in assignedTo is a free text officer name. It is not a database relationship to a user account and does not restrict an officer to assigned assets. Both authenticated roles can read all departments. The movement simulator is visible to officers, but the backend rejects their mutation requests.

Scope boundaries

The current application covers a single monitoring station and a local web interface. It does not include automatic GPS tracking, radio signal distance estimation, live map coordinates, distributed sensor gateways, approval based transfers, email or SMS delivery, or a government identity provider. The project name describes the intended asset use case and does not imply government certification or production approval.

2 Architecture and application flow

The packaged application runs one Spring Boot process on the laptop. It serves the built React files and the REST API on the same origin. This keeps browser session cookies and CSRF handling within a single local application.

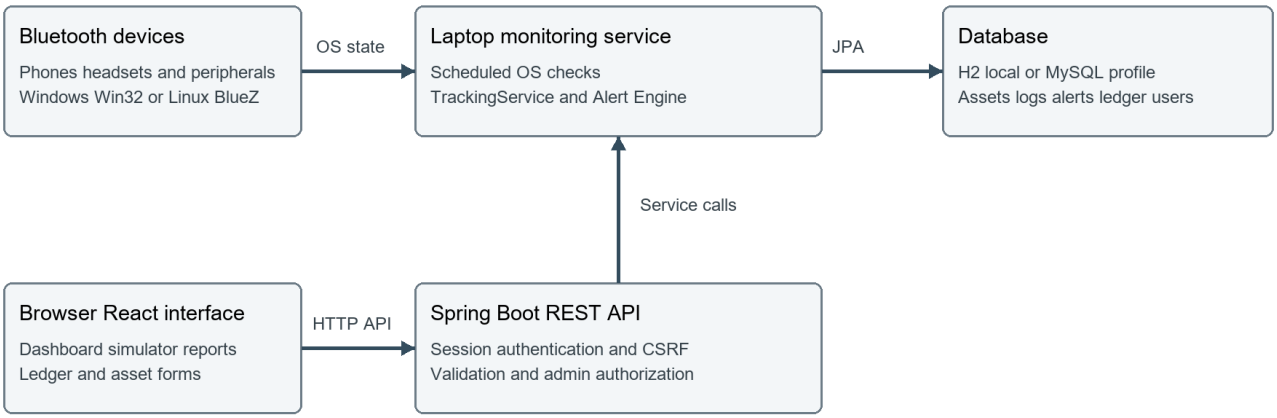


Figure 1 Application components and data flow

Browser request path

React checks /api/session before rendering protected pages. It obtains JSON through the Fetch API and sends a fresh CSRF header before mutations. Spring Security authenticates the session and enforces admin permissions. ApiController validates input records and calls TrackingService. Repositories persist the resulting asset state, logs, alerts and ledger blocks.

Monitoring path

BluetoothMonitor runs on a fixed delay schedule and executes an operating system command for each asset. A successful probe produces a connected or disconnected observation. TrackingService applies a state change, records the log, creates or resolves alerts and appends the relevant ledger event. The browser learns about changes through polling rather than a server push channel.

Data and transaction boundaries

TrackingService uses Spring transactions and a pessimistic database row lock when updating an existing asset. Repeated observations of the same status return without adding another transition log. Logs and alerts store asset identifiers and names as snapshots rather than using JPA entity relationships, allowing history to remain after deletion.

Development and packaged execution

During development, Vite normally serves React on port 5173 and proxies /api to the backend on port 8080. For packaging, Vite writes frontend/dist. Maven copies that output into the JAR static resources. Node.js is needed to build the interface, but a separate Node server is not needed to run the finished JAR.

3 Technology stack and dependencies

The versions below distinguish the declared backend baseline from frontend versions resolved in package-lock.json. Dependency upgrades can change the locked frontend versions; the lock file is the reproducibility reference.

Layer	Technology	Current version or role
Language	Java	Maven compilation target 17
Backend framework	Spring Boot	3.5.7 from pom.xml
HTTP and API	Spring Web and embedded Tomcat	REST controllers and static React hosting
Persistence	Spring Data JPA and Hibernate	Repository operations and entity mapping
Security	Spring Security and BCrypt	Sessions roles password hashes and CSRF
Validation	Jakarta Validation	Required fields size limits and MAC format
Scheduling	Spring Scheduling	@Scheduled background monitor
Local database	H2	File database under data/assets
Optional database	MySQL and Connector J	Enabled using the mysql Spring profile
Frontend	React and React DOM	19.3.1 resolved in lock file
Routing	React Router DOM	7.18.5 resolved
Bundler	Vite and React plugin	7.3.7 and 5.2.0 resolved
Web presentation	HTML CSS JavaScript JSX SVG	Custom styles icons Fetch Web Audio and Notifications APIs
Frontend tests	Vitest jsdom Testing Library	Vitest 3.2.7 jsdom 26.1.0 React testing 16.3.3
Backend tests	Spring Boot Test and Security Test	JUnit assertions and MockMvc integration tests
Windows alerts	WinRT and Windows Forms	Native toast with balloon fallback plus system sound
Build tools	Maven npm PowerShell	JAR packaging locked install and build scripts
OS integration	PowerShell C sharp Win32 and BlueZ	bthprops.cpl P Invoke on Windows; bluetoothctl on Linux
Ledger algorithm	Java MessageDigest SHA-256	Hash linkage and a small proof of work loop

Spring Boot manages most Java dependency versions through its parent configuration. The project does not currently use JSP, JSTL, Jasper, Bootstrap, BlueCove, PyBluez, Bleak, WebSockets, a GPS service, Ethereum or Hyperledger. The cryptographic ledger is implemented directly in Java and stored in the application database.

4 Interface screens and implemented features

Screen or route	What the user can do
/login	Sign in using username and password. Login errors are shown in place. Notification permission is requested after successful sign in when supported.
/ overview	View registered assets connected count open alerts ledger count and a security banner. Search records and inspect discovered Bluetooth devices; admins see import controls.
/assets/new	Register a device with serial number MAC department officer current location and permitted places.
/assets/{id}/edit	Update asset metadata and permitted zones. The existing Bluetooth address is read only.
/simulation	Select an asset choose a predefined zone or type a custom destination. Show permitted and restricted choices and the transfer result.
/blockchain	Inspect blocks newest first including action path authorization hashes timestamp nonce and operator. Request an integrity audit.
/reports	Inspect the latest 100 connection and location events with browser local timestamps.

Live updates and interaction behavior

The shared useResource hook requests data immediately. The dashboard summary polls every 1500 ms and discovered Bluetooth devices every 2000 ms. Other pages using the true polling flag default to 1500 ms. It prevents overlapping requests within the active polling effect, stops polling when unmounted and retains the last successfully loaded data if a refresh fails. Dashboard search and form data remain in React state instead of being cleared by a full page reload.

Admins see edit, delete and shift controls. Manual connection buttons appear only when the backend reports demo mode. Asset deletion uses a browser confirmation. Input fields show validation feedback and retain entered values when a request is rejected. The application has responsive CSS layouts and uses inline SVG icons.

Browser notification behavior

When the dashboard open alert count increases, it attempts a two tone Web Audio chime and, if permission is granted, a browser notification with the newest unresolved alert. A visible security banner appears while alerts remain open. Separately, the backend starts a Windows notification helper when it creates a connection loss alert. Browser and operating system settings affect delivery; these channels have different triggers and are described in Section 18.

5 Main operating workflows

Register and manage an asset

An admin signs in, opens Register asset and submits the identity and assignment fields. The manual create API validates the MAC format and required values, then stores an UNKNOWN connection state. Serial numbers and Bluetooth addresses are unique. Registration appends an ASSET_REGISTERED block. Editing appends ASSET_UPDATED; deletion closes open alerts, writes ASSET_DELETED and retains existing history records.

Observe loss and reconnection

- The monitor or demo button submits a new observation. Identical consecutive states are ignored.
- On a transition to DISCONNECTED, the service writes a tracking log. If the asset has no unresolved alert, it creates a connection alert, a STATUS_ALERT block and starts the Windows notification helper when its script exists.
- On a transition to CONNECTED, the service writes a log, resolves open alerts for the asset and writes a STATUS_CHANGE block.
- The dashboard discovers the change during a later poll and may display a browser notification if the open alert count increases.

The first observation may be disconnected and still create an alert. The current reconnection code resolves all open alerts, including location alerts. This is a known correctness issue described in Section 15.

Submit an authorized location transfer

The admin selects an asset and a destination in the simulator. The backend splits allowedLocations on commas or semicolons and compares trimmed names without case sensitivity. A permitted move updates currentLocation, records a relocation log and appends LOCATION_SHIFT. Matching movement breach alerts are resolved when the asset returns to a permitted zone.

Submit an unauthorized location transfer

A destination outside the list still becomes the recorded current location. The system creates a security breach alert and a SECURITY_BREACH_ALERT block with authorized set to false. It reports the result to the simulator. The operation records an unauthorized event; it does not physically stop equipment from moving or reject the location update.

Example demonstration

Register a demonstration phone with HQ Main Facility and Dispatch Office as allowed zones. In demo mode, connect and disconnect it to show the connection alert. Reconnect it, then move it to Dispatch Office to show an authorized transfer. Move it to External Off-Site to show a breach. Review the tracking history and ledger, run the cryptographic audit, and return it to a permitted zone. Use test records rather than real operational assets for this walkthrough.

6 Database design and asset records

JPA entities define five tables. Hibernate updates the schema on startup through spring.jpa.hibernate.ddl-auto=update. H2 is the default file database; the MySQL profile uses the same entity model. No versioned database migration scripts are provided.

Table	Purpose	Logical association
users	Login credentials and role	Independent user accounts
assets	Device identity assignment location and status	Referenced by logs alerts and ledger through assetId
tracking_logs	Connection and movement observations	Asset identifier and name snapshot
alerts	Connection and movement warnings	Asset identifier and name snapshot
blockchain_blocks	Linked cryptographic event records	Optional asset identifier and name snapshot

Users

Field	Type and rule
id	Generated Long primary key
username	Unique non null String
password	BCrypt encoded password String
role	String ADMIN or OFFICER used by the security configuration

Assets

Field	Type and validation
id	Generated Long primary key
name	Required String up to 100 characters
type	Required String up to 50 characters
serialNo	Required unique String up to 100 characters
bluetoothMac	Required unique colon separated six byte hex address; uppercase on save
department	Required String up to 100 characters
assignedTo	Required free text officer name up to 100 characters
currentLocation	String up to 150 characters; default HQ Main Facility
allowedLocations	String up to 500 characters; comma or semicolon separated names
status	String enum UNKNOWN CONNECTED or DISCONNECTED

Allowed zones are stored as one text field rather than a normalized zones table. Facility zone choices come from a hard coded API list. The frontend default includes Server Room B-4, while the backend fallback includes Server Room; these are different exact zone names and can produce different authorization results.

7 Event alert and ledger records

Tracking logs

Field	Meaning
id	Generated Long identifier
assetId and assetName	Asset snapshot identifying the observed device
status	UNKNOWN CONNECTED or DISCONNECTED at the time of logging
timestamp	Java Instant for the event time
source	Probe source or relocation details including old and new named zones

Connection changes create logs only when the status changes. Location shifts also create logs, even when the connection state stays the same. The reports endpoint returns the most recent 100 entries, ordered by timestamp descending.

Alerts

Field	Meaning
id	Generated Long identifier
assetId and assetName	Asset snapshot
message	Human readable warning describing connectivity loss or unauthorized movement
createdAt	Java Instant when the alert was created
resolved	Boolean indicating whether the warning is closed
resolvedAt	Instant set when an alert is resolved

There is currently no separate alert category, severity, acknowledgement, resolution reason or resolver field. Movement alert resolution relies on text matching within message. The summary contains the latest 100 alerts and a count of all unresolved alerts.

Blockchain blocks

Field group	Stored values
Identity and ordering	Generated id and Long blockIndex
Cryptographic linkage	64 character previousHash and hash
Time and work	timestamp and nonce
Event and asset	action up to 50 characters assetId and assetName
Movement and actor	fromLocation toLocation and operator
Payload and outcome	details up to 500 characters and boolean authorized

The ledger repository supports newest first and oldest first ordering and asset based lookup. The current API returns the entire ledger rather than a paginated subset. blockIndex is non null but is not declared unique at the entity level. Asset names are stored in blocks but are not included in the current hash calculation.

8 Session and read API reference

The browser uses same origin HTTP requests. Except for the session endpoint and authentication flow, API reads require an authenticated account. Static React page routes are public shells; protected data is retrieved separately from the API.

Method and path	Response and use
GET /api/session	200 JSON with authenticated username admin csrfToken and csrfHeader. Also works before login.
POST /api/login	Username and password as form URL encoded fields plus CSRF header. Returns 204 on success or 401 on rejection.
POST /api/logout	Invalidates the authenticated session with CSRF protection. Returns 204.
GET /api/summary	Assets latest 100 alerts total openAlerts monitoring mode and totalBlocks.
GET /api/logs	Latest 100 TrackingLog objects newest first.
GET /api/assets/{id}	Asset object or 404 when not found.
GET /api/facility-zones	List of predefined named zones for the simulator.
GET /api/bluetooth/devices	Windows scan objects with Name Mac Connected Remembered and DeviceClass; scan failures currently return an empty list.
GET /api/blockchain	All blocks newest first and a chain verification result.
GET /api/blockchain/verify	valid totalBlocks latestHash and an optional error description.

Session lifecycle

React fetches /api/session when the app starts. It redirects unauthenticated visitors to /login and preserves their intended page path. On success it obtains the new session and CSRF token. Before every mutation, the request helper fetches a fresh token and sends it under the header name returned by the server. This also handles the token change that follows login and logout.

Read responses and error handling

Unauthorized API access returns 401. The frontend request helper dispatches a session expired event and the protected layout returns to login. A 403 response indicates a denied operation or CSRF failure. JSON validation errors include a message and field error map. Duplicate database values are reported as 409. Server error messages are suppressed by default outside the explicit API error handler.

GET /api/blockchain performs verification as well as loading all blocks. With 1500 ms polling on the explorer, this creates repeated work proportional to the size of the ledger. A larger deployment needs pagination, summary endpoints and a deliberate audit schedule.

9 Asset mutation API reference

Asset mutations require ADMIN and a valid session CSRF token. The Bluetooth sync endpoint is an exception: the current security rules permit any authenticated user, even though the interface shows import controls only to admins. Asset DTOs prevent clients from assigning observed status or choosing creation identifiers.

Method and path	Body	Successful result
POST /api/assets	Asset fields as JSON	201 Created; empty body
PUT /api/assets/{id}	Asset fields as JSON	204 No Content
DELETE /api/assets/{id}	No body	204 No Content; history retained
POST /api/assets/{id}/simulate/{status}	No body; status in path	204 No Content; demo mode only
POST /api/assets/{id}/shift	JSON with toLocation	200 JSON with asset authorized alertCreated message and block
POST /api/bluetooth/sync	No body	200 JSON array of newly imported devices; current access is authenticated

Asset input example

```
{
  "name": "Demonstration phone",
  "type": "Phone",
  "serialNo": "DEMO-001",
  "bluetoothMac": "AA:BB:CC:DD:EE:01",
  "department": "Public Works",
  "assignedTo": "Demonstration Officer",
  "currentLocation": "HQ Main Facility",
  "allowedLocations": "HQ Main Facility, Dispatch Office"
}
```

Transfer input example

```
{ "toLocation": "Dispatch Office" }
```

The transfer destination is required and limited to 150 characters. It may be a custom zone rather than an item in the predefined list. The backend classifies it against the asset permitted places. An unauthorized transfer still returns a successful request response with authorized false and alertCreated true, because recording the breach is the intended operation.

Important mutation rules

- Serial numbers and MAC addresses must be unique. An existing Bluetooth address is immutable.
- Connection simulation accepts CONNECTED or DISCONNECTED only and is rejected outside demo mode.
- Location transfer is not restricted to demo mode; an admin can submit one while Windows or Linux monitoring is active.
- Missing optional location fields fall back to backend defaults, including on update. Clients should send the existing values when retaining them.
- A duplicate integrity violation is reported as a duplicate serial or MAC even if another database constraint caused the failure. Error classification needs refinement.

10 Bluetooth monitoring implementation

Mode	Probe and current behavior
windows default	Runs scripts/scan-bluetooth.ps1 through PowerShell. Embedded C sharp calls BluetoothFindFirstDevice and BluetoothFindNextDevice in bthprops.cpl. The scanner returns the native fConnected value along with identity and class data.
linux	Runs bluetoothctl info for each registered MAC. A successful response with Connected yes becomes CONNECTED; another valid Connected value becomes DISCONNECTED.
demo	Disables scheduled OS polling. Admin controls submit manual connection transitions. The Windows discovery endpoint is not disabled by demo mode and still invokes its platform script.

Timing and scope

The scheduler waits 2000 ms after each monitoring batch. A Windows batch performs one full host scan with a ten second timeout, imports newly connected devices and compares registered MACs with the scan. Linux probes run sequentially with a three second timeout per asset. Browser summary polling is separate at 1500 ms. These are scheduling settings, not a guaranteed detection latency.

Windows observations

A nonempty Windows scan becomes a MAC lookup map. Known records matching a connected device become CONNECTED; registered records absent from that nonempty map become DISCONNECTED. The scanner asks for connected, remembered, authenticated and unknown device records but sets fIssueInquiry to false. It enumerates records known to Windows; it does not actively search for every nearby radio or calculate distance.

Empty results and stale states

A missing script, timeout, malformed output or caught exception returns an empty device list. The Windows monitor immediately returns for an empty list, preserving all previous asset states. Consequently a genuine zero device result is indistinguishable from a failed scan and can leave the last device marked CONNECTED. A one device PowerShell result can also be serialized as a JSON object, while Java currently accepts only a JSON array. Check exit codes, output shape and scan health explicitly.

Hardware and deployment boundaries

A native connected flag is stronger evidence than the earlier PnP health value, but it still represents an OS connection observation rather than verified room or GPS location. Test the actual phone, active profile, radio off behavior and OS update latency. The external scanner script is resolved relative to the process working directory and is not included by the Maven resource configuration; deploy scripts alongside the JAR or configure an explicit path.

Linux exceptions preserve the last state. Monitoring does not measure signal strength or meters. Nearby unpaired devices and Bluetooth Low Energy advertisements are not comprehensively discovered by this classic Windows enumeration path.

11 Local cryptographic audit ledger

Purpose and event coverage

BlockchainService stores a local sequence of blocks in blockchain_blocks. On initialization it creates a GENESIS record if the repository is empty. Asset registration, metadata updates, deletion, authorized transfers, movement breaches and selected connection changes append blocks. This provides an inspectable event chain for the project demonstration.

Action	When it is appended
GENESIS	Initial ledger setup
ASSET_REGISTERED and ASSET_UPDATED	Creation and metadata editing
ASSET_DELETED	Deregistration of an asset
LOCATION_SHIFT	A transfer to a permitted zone
SECURITY_BREACH_ALERT	A transfer to a zone outside the permitted list
STATUS_ALERT	Disconnection creates a new connection alert
STATUS_CHANGE	A transition to CONNECTED

Hash construction and proof of work

For each block, the service combines the block index, previous hash, timestamp in epoch milliseconds, action, asset identifier, source location, destination location, operator, details, authorized flag and nonce using colon separators. Java MessageDigest computes SHA-256 and HexFormat creates the hexadecimal string. Mining increments a nonce until the hash begins with one zero, subject to a one million iteration cap. This is a deliberately small demonstration workload.

Verification behavior

The audit reads blocks in ascending index order, recalculates their hashes, compares each previousHash with the preceding block hash and checks that the first previousHash is 64 zeros. It returns whether those checks passed, how many blocks were read, the latest hash and any detected error. The explorer displays these results and permits a manual verification request.

Meaning and limits of a passed check

A passed check establishes consistency under the current algorithm for the data read. It does not prove that a location is physically correct, that an operator signed an event, or that a database administrator could not rewrite the chain. There are no distributed peers, consensus protocol, digital signatures or external trusted checkpoints.

assetName is excluded from the hash. Verification does not enforce the mining prefix or sequential block indexes, and a valid tail removal can remain undetected. The delimiter based input is not a canonical structured encoding. Ledger append synchronization is local to one JVM; the schema has no unique blockIndex constraint or explicit cross process chain lock. These are improvement items before stronger integrity claims.

12 Authentication security and validation

Account and session model

Spring Security loads usernames, BCrypt password hashes and roles from the users repository. Bootstrap now upserts admin and officer on every startup and replaces their password hashes. ADMIN_PASSWORD and OFFICER_PASSWORD override the password; when blank, each password defaults to its username. Restarting without overrides therefore resets these built in accounts to weak predictable credentials. Configure strong secrets and change this initializer before shared deployment. Existing account roles are not reset by the upsert.

The session cookie is HttpOnly and SameSite=Lax. CSRF protection remains enabled for login, logout and mutations. Authentication failure returns 401; admin authorization is enforced on /api/assets/** mutations independently of whether the interface hides a control. Static React files contain no embedded protected records.

Validation and data handling

- Required text fields and maximum lengths are checked with Jakarta Validation. MAC addresses must use six colon separated hexadecimal bytes.
- The asset input DTO accepts identity assignment and location configuration only. Observed status and generated identifiers are not writable creation fields.
- React renders data through normal text expressions. Form failures are shown inline and entered values are retained.
- Database uniqueness protects the registered serial number and Bluetooth identity. Asset row locking reduces conflicting updates within tracking transactions.

Access boundaries and operational privacy

Officer asset management is intended to be read only across all departments, but the Bluetooth sync mutation currently remains accessible to any authenticated account. There is no department filter, user management screen, self service password reset, multi factor authentication, login rate limiter or external identity integration. The ledger contains actor labels, but registration, update and deletion currently hard code admin and monitor actions use monitor. Those labels are not signed identity evidence.

The server binds to 127.0.0.1 by default. Exposing the system beyond the laptop requires an explicit deployment design for HTTPS, secure cookies, authorization scope, secrets, logs, backups and database permissions. H2 uses a local file and an empty default database password; protect the data directory and avoid treating this configuration as a shared production service.

Notification privacy

Notifications can show an asset name and alert description on the desktop. Permission depends on the user browser settings and should be explained during demonstrations. Browser notifications are generated from dashboard polling. Native desktop notifications are spawned by the backend on the monitoring laptop, so they may also appear without an open dashboard. A future design should allow users to opt in deliberately and control message detail.

13 Build configuration and operation

Build and package

Use the project build script after installing the required Java, Maven, Node.js and npm tools. The script installs dependencies from the lock file, runs frontend tests, builds React, then runs Maven clean package. The compilation target is Java 17. Choose a JDK supported by the selected Spring Boot release for deployment; the README also records prior use of Java 26.

```
./scripts/build.ps1
java -jar target/asset-tracker-1.0.0.jar
```

Run from the project root so scripts/scan-bluetooth.ps1 and scripts/notify-windows.ps1 can be found. The JAR alone does not contain those helpers. The interface is available at http://localhost:8080. To build manually:

```
npm --prefix frontend ci
npm --prefix frontend test
npm --prefix frontend run build
mvn clean package
```

Development workflow

Run the backend first, then start Vite using npm --prefix frontend run dev. The normal frontend URL is http://127.0.0.1:5173. The Vite proxy forwards /api requests to 127.0.0.1:8080. Rebuild frontend/dist before creating a new backend package; Maven does not run npm automatically.

Configuration inputs

Setting	Use
TRACKING_MODE	windows default; linux for BlueZ; demo for manual connection observations
tracking.interval-ms	Backend monitor fixed delay; currently 2000
ADMIN_PASSWORD and OFFICER_PASSWORD	Passwords reapplied to built in accounts at every startup
DB_URL DB_USERNAME DB_PASSWORD	MySQL connection details when the mysql profile is active
spring.profiles.active=mysql	Selects MySQL instead of the default H2 file configuration

Local data and repeatable operation

The default database persists under data/assets relative to the working directory. Launching from another folder may create or use another database. Stop the application before copying an H2 file backup or cleaning build outputs on Windows. The Maven clean plugin is configured not to fail the build on cleanup errors, so a successful package can still contain stale output unless the result is checked. Prefer a clean checkout or an isolated build directory for repeatable delivery.

The scripts folder includes the runtime scanner and desktop notifier plus registration, API and Bluetooth diagnostic helpers. Several helpers embed a specific device or login assumption; parameterize them before reuse. Do not stop all Java processes just to restart this application: identify its process specifically. Do not publish real device identifiers or credentials in reports.

14 Testing and demonstration acceptance

Automated test definitions

The current source contains 12 Java test methods and 6 React component tests. These definitions document intended behavior; a current clean execution is required before making a release pass claim. This report does not treat previous test results from an earlier project version as proof of the newer ledger and Windows behavior.

Test layer	Covered scenarios
Java service and API tests	Idempotent observations reconnection resolution retained history authentication anonymous CSRF token officer restrictions CSRF enforcement MAC validation status binding and deep link forwarding.
Java location and ledger test	A permitted shift an unauthorized shift alert creation and a valid chain after those operations.
React component tests	Login dashboard polling without clearing search officer controls preservation of input after a duplicate error movement breach feedback and ledger audit interaction.

Known test configuration gap

The officer read test expects summary.mode to equal demo, but application.properties defaults to windows and the test annotation does not explicitly override tracking.mode. The suite should pin demo mode so it is independent of machine environment and cannot invoke OS probes unexpectedly.

Recommended acceptance checks

- Use an isolated in memory test database. Verify fresh account initialization login logout expired session behavior and CSRF rotation.
- Register edit search and delete a test asset. Confirm duplicate serial and MAC rejection and immutable Bluetooth identity.
- In demo mode observe UNKNOWN then CONNECTED then DISCONNECTED then CONNECTED. Confirm no duplicate transition records.
- Shift to a permitted zone then to a restricted zone. Check logs alerts and ledger records; return the asset to an allowed zone.
- Reproduce reconnect while a location breach is open. The breach must remain open until its location condition is resolved after the alert logic is corrected.
- Check ledger validation after simulated data tampering, name edits tail removal concurrent appends and invalid index or proof of work values.
- Test the actual Windows phone with Bluetooth enabled disabled powered off and physically moved. Compare OS observations with the UI rather than relying on pairing alone.

MySQL connectivity, zero and single device scans, import authorization, account reset behavior, hardware accuracy, notification delivery and deployment beyond localhost need environment specific acceptance testing. Performance and recovery behavior also require explicit tests before using more than a small demonstration dataset.

15 Limitations and development priorities

Priority	Finding	Recommended next action
High	Empty or single device scans can preserve stale states. Missing scripts and scan failures share an empty response.	Return explicit probe health support zero one and many devices check exit codes and test OS behavior.
High	Bluetooth sync permits authenticated officers; startup resets passwords to predictable defaults without overrides.	Enforce ADMIN on sync and change initialization to preserve stored passwords or require explicit reset.
Medium	Import sets input status but save does not copy it. A deleted connected device is auto registered again.	Route initial observations through the transition service and define excluded device or decommissioning rules.
High	Reconnection resolves every open alert for an asset, including a location breach. An open movement alert can suppress creation of a connection alert.	Add explicit alert categories and independent conditions for creation deduplication and resolution.
High	Current location can be edited directly through asset save without the relocation authorization workflow.	Separate configuration updates from movement events or apply the same movement checks to location edits.
Medium	Ledger claims exceed the implemented integrity checks. Actor labels are unsigned and some fields are unhashed.	Use canonical payload encoding hash all required fields add sequence and work checks and anchor trusted checkpoints.
Medium	Ledger appends are synchronized only within one process, and blockIndex has no uniqueness constraint.	Define database level append locking and ordering before supporting multiple monitoring stations.
Medium	Location defaults differ between frontend and backend, and free text zones can drift.	Normalize zones and permitted asset zone associations with consistent identifiers.
Medium	Probe health is not visible. Ledger retrieval and verification repeat every 1500 ms; UI and scheduler may launch concurrent scans.	Add health indicators cache scan results paginate blocks and separate automatic polling from full audit work.
Medium	Some messages can exceed default database column lengths. All integrity violations are labeled as duplicate identity.	Align validation and column lengths for logs alerts and block details; classify failures precisely.

Planned enhancements rather than current features

Future work can include GPS or BLE gateway inputs, department scoped access, maintenance and lifecycle records, email or SMS alerts, map views, approval based transfers, report export, paginated audit queries and centralized deployment. These require additional implementation and validation; they are not capabilities of the present application.

The immediate development goal should be a reproducible demonstration with truthful connection and integrity labels. Correctness of observations and alert resolution is more important than adding more dashboard features.

16 Implementation map and terminology

Source map

Source area	Responsibility
pom.xml	Spring Boot version Java target dependencies JAR resources and clean plugin behavior
frontend/package.json and package-lock.json	Declared and resolved React build and test dependencies
frontend/src/App.jsx	Login protected layout overview asset forms simulator ledger explorer and reports
frontend/src/api.js	Fetch helper fresh CSRF headers and expired session handling
frontend/vite.config.js	React plugin build output and local API proxy
ApiController and ApiExceptionHandler	JSON endpoints validation DTOs and structured errors
SecurityConfig and Bootstrap	Session security role rules password hashing and initial accounts
TrackingService and BluetoothMonitor	Business rules database transactions movement classification and OS probes
BlockchainService Block BlockRepository	Genesis append hashing proof of work persistence and verification
Entity and repository classes	Users assets logs alerts database fields and query ordering
application.properties and mysql profile	Local defaults database profile monitor mode and environment variables
ApplicationTest and App.test.jsx	Backend integration and frontend component test definitions
scripts/build.ps1 and register-phone.ps1	Build orchestration and device specific registration helper
scripts/scan-bluetooth.ps1	Embedded C sharp Win32 device enumeration and JSON output
scripts/notify-windows.ps1 and diagnostic helpers	Native toast sound balloon fallback and OS or API investigation

Java source names in this table are located under src/main/java/gov/assets. Java tests are under src/test/java/gov/assets and React tests under frontend/src. The current code takes precedence where README claims conflict with implementation.

Terms used in the project

Asset means a registered piece of equipment. Monitoring station means the laptop running Spring Boot and the OS probe. Observation means the state reported by a probe or simulation input. Facility zone means a named place submitted to the movement API. Breach means a recorded move outside an asset permitted list. CSRF token means a session linked value checked on state changing requests. Hash chain means records linked through cryptographic hashes. Proof of work nonce means the changing value used to find a hash with the configured prefix.

Document baseline

Project version 1.0.0. Code and dependency snapshot dated 30 October 2026. This document describes the implemented local demonstration and the improvements needed before stronger operational guarantees. It intentionally contains no account passwords, real phone identifiers or personal device registration values.

17 Bluetooth discovery and registry import

Discover devices on the monitoring laptop

The dashboard calls GET /api/bluetooth/devices every two seconds. BluetoothMonitor launches scripts/scan-bluetooth.ps1 with PowerShell in noninteractive mode. The script compiles embedded C sharp with Add-Type and uses P Invoke against bthprops.cpl. It closes the enumeration handle in a finally block and emits JSON for Java Jackson parsing.

Scan field	Meaning
Name	Name known to the Windows Bluetooth stack
Mac	Six byte hardware address formatted with colons
Connected	Native BLUETOOTH_DEVICE_INFO fConnected flag
Remembered	Whether Windows remembers the device
DeviceClass	Bluetooth class of device numeric value

Import and automatic registration

Sync all to registry calls POST /api/bluetooth/sync. The service normalizes scanned and registered MACs by removing separators and compares them case insensitively. A matching asset is skipped; an unmatched device is registered. This import includes disconnected remembered devices returned by the scan, not just connected devices. The per row Track control calls the same bulk sync operation rather than importing only the selected row.

In Windows monitoring mode, the scheduler also auto registers any connected scanned device that is not in the registry. It then submits connection observations for all registered assets. Type inference uses device name keywords and the major Bluetooth device class, with categories such as smartphone, audio or headset, peripheral, wearable and generic Bluetooth device.

Generated field	Import value
serialNo	BT followed by the normalized MAC
department	Connected Peripherals
assignedTo	Laptop Bluetooth Host
currentLocation	Laptop Workstation
allowedLocations	Laptop Workstation Office and Desk

Import edge cases and authority

The import path assigns status on its input object, but TrackingService.save creates a fresh entity and does not copy that field. Manual sync records therefore initially remain UNKNOWN until an observation is processed; auto discovery is followed by a monitoring pass. Import bypasses the REST creation DTO, although entity constraints and database rules still apply. Deduplication repeatedly loads the registry and concurrent import requests can race. New registrations append ordinary ASSET_REGISTERED ledger blocks with a hard coded admin operator label.

18 Alert delivery and notification behavior

Stored alerts and visible warning banner

Alerts persist in the database and feed the summary open alert count. The dashboard displays a warning banner while the count is positive, showing up to two unresolved messages from the recent alert list. The list below it includes recent open and resolved records. An initial disconnected observation can create an alert; identical status observations do not create duplicate transition logs.

Browser sound and notification

When the open count rises, playAlarmChime creates a Web Audio context and generates two sawtooth tones, approximately 880 Hz for 0.15 seconds and 587 Hz for 0.35 seconds. Browser autoplay policy may prevent sound until user interaction. If Notifications permission is granted, the dashboard also sends a popup using the newest unresolved alert. The same tag replaces the preceding popup. The login flow requests permission when the browser still reports its default state.

The counter starts at zero on dashboard mount, so existing open alerts can trigger a notification on first load. A new alert can be missed when another alert resolves between polls and the total count does not increase. Movement alerts can trigger the same channel. These are count based alerts rather than guaranteed per event deliveries, and they require the dashboard to remain mounted.

Native Windows desktop channel

When TrackingService creates a new disconnection alert, it starts scripts/notify-windows.ps1 if the script exists. The helper plays the system Exclamation sound and attempts a Windows Runtime toast. If toast creation fails, it tries a Windows Forms NotifyIcon balloon and then disposes it. The service does not wait for delivery or record success, and it ignores startup exceptions. Desktop policy, user session state and script availability can prevent a popup.

This native path is attached to connection alert creation only, not directly to movement breach creation. It is invoked inside the database operation before the transaction commits, so a notification is not proof that the transaction ultimately succeeded. The helper is not explicitly gated by tracking mode or OS; on an unsupported host process startup can fail silently.

Operational expectations

Dashboard and native channels can both fire for one connection loss, producing two sounds or popups. Notification text can expose asset details on the laptop desktop. XML text in the native toast is assembled from unescaped title and message strings, so special characters in an asset name can break toast parsing and invoke the fallback. Treat all delivery as best effort until tested. There is no email SMS background web push delivery receipt or acknowledgement workflow.